

ДПО	Фронтенд и бэкенд разработка
ДИСЦИПЛИНА	Основы фронтенд-разработки
ВИД УЧЕБНОГО МАТЕРИАЛА	Методические указания к практическим занятиям

Практическое занятие 14 (Vue): Создание простого приложения на Vue 3

Цель занятия

Научиться создавать Vue-приложение с нуля, освоить основные концепции: компоненты, пропсы, реактивное состояние, обработку событий и условный рендеринг. В результате будет создано работающее приложение - **персональный менеджер книг для чтения** с возможностью добавления, отметки прочитанных, оценки и удаления. Также рассмотрим процесс публикации Vue-приложения на GitHub Pages.

Теоретическая справка

Как создается Vue-приложение

Современный способ создания Vue-приложений - использование инструмента Vite, с которым мы ознакомились ранее.

Команда для создания проекта:

```
npm create vite@latest book-manager -- --template vue
```

После создания нужно перейти в папку проекта, установить зависимости и запустить сервер разработки:

```
cd book-manager
npm install
npm run dev
```

Структура проекта

Созданный проект имеет следующую структуру ключевых файлов:

- `src/main.js` - точка входа, здесь создается и монтируется Vue-приложение.
- `src/App.vue` - корневой компонент приложения (однофайловый компонент).
- `src/components/` - папка для дочерних компонентов.
- `index.html` - HTML-страница с контейнером `<div id="app">`.

Основные понятия Vue

Компонент - строительный блок интерфейса. В Vue 3 компоненты обычно создаются в формате однофайловых компонентов (SFC - Single File Component), которые содержат шаблон (template), логику (script) и стили (style) в одном файле.

```
<template>
  <h1>{{ message }}</h1>
</template>

<script setup>
const message = 'Привет, мир!'
</script>
```

Директивы - специальные атрибуты с префиксом **v-**, которые добавляют динамическое поведение:

- **v-bind** или **:** - связывание данных с атрибутами.
- **v-on** или **@** - обработка событий.
- **v-if**, **v-else-if**, **v-else** - условный рендеринг.
- **v-for** - рендеринг списков.
- **v-model** - двустороннее связывание для форм.

Реактивное состояние создается с помощью функции **ref()** или **reactive()**.

```
<script setup>
import { ref } from 'vue'

const count = ref(0) // ref для примитивов

const increment = () => {
  count.value++ // обращение через .value
}
</script>
```

Вычисляемые свойства (computed) используются для производных состояний, которые кэшируются и обновляются только при изменении зависимостей.

```
<script setup>
import { ref, computed } from 'vue'

const price = ref(100)
const quantity = ref(2)
const total = computed(() => price.value * quantity.value)
</script>
```

Пропсы (props) - входные данные, которые родительский компонент передает дочернему.

```
<!-- Дочерний компонент -->
<script setup>
defineProps(['title', 'author'])
</script>

<template>
  <h3>{{ title }}</h3>
  <p>Автор: {{ author }}</p>
</template>
```

События (emits) - механизм передачи сообщений от дочернего компонента к родительскому.

```
<!-- Дочерний компонент -->
<script setup>
const emit = defineEmits(['book-selected'])

const selectBook = () => {
  emit('book-selected', bookId)
}
</script>
```

Разбор примеров

Пример 1: Счётчик книг

Простейший компонент для подсчета количества книг.

```
<template>
  <div class="counter">
    <h2>Книг в коллекции: {{ count }}</h2>
    <button @click="increment">+ Добавить книгу</button>
    <button @click="decrement" :disabled="count === 0">- Убрать книгу</button>
    <p v-if="count === 0" class="hint">Добавьте первую книгу!</p>
    <p v-else-if="count >= 10" class="hint">Отличная коллекция!</p>
  </div>
</template>

<script setup>
import { ref } from 'vue'

const count = ref(0)

const increment = () => {
  count.value++
}

const decrement = () => {
```

```
    if (count.value > 0) {
      count.value--
    }
  }
</script>

<style scoped>
.counter { text-align: center; }
.hint { color: #666; font-style: italic; }
button:disabled { opacity: 0.5; cursor: not-allowed; }
</style>
```

Как это работает:

- `ref(0)` создает реактивное состояние.
- `@click` обрабатывает клики по кнопкам.
- `v-if/v-else` показывают подсказки.
- `:disabled` динамически блокирует кнопку.

Пример 2: Поиск книги по названию

Компонент с двусторонним связыванием и вычисляемым свойством.

```
<template>
  <div>
    <h2>Поиск книги</h2>
    <input
      v-model="searchQuery"
      type="text"
      placeholder="Введите название..."
    />
    <p v-if="searchQuery">
      {{ searchResult }}
    </p>
  </div>
</template>

<script setup>
import { ref, computed } from 'vue'

const searchQuery = ref('')
const books = ref(['Война и мир', 'Преступление и наказание', 'Анна Каренина'])

const searchResult = computed(() => {
  if (!searchQuery.value) return ''

  const found = books.value.find(book =>
    book.toLowerCase().includes(searchQuery.value.toLowerCase())
  )

  return found
})
```

```
    ? `Найдена книга: ${found}`  
    : 'Книга не найдена'  
  })  
</script>
```

Как это работает:

- **v-model** связывает поле ввода с состоянием **searchQuery**.
- **computed** автоматически обновляет результат поиска.
- При изменении поискового запроса результат пересчитывается.

Пример 3: Рейтинг книги

Компонент для оценки книг звездочками.

```
<template>  
  <div class="rating">  
    <span v-for="star in 5" :key="star" class="star" @click="setRating(star)">  
      {{ star <= modelValue ? '★' : '☆' }}  
    </span>  
    <span v-if="modelValue" class="rating-text">({{ modelValue }}/5)</span>  
  </div>  
</template>  
  
<script setup>  
defineProps(['modelValue'])  
defineEmits(['update:modelValue'])  
  
const setRating = (value) => {  
  emit('update:modelValue', value)  
}  
</script>  
  
<style scoped>  
.star {  
  font-size: 24px;  
  cursor: pointer;  
  color: gold;  
}  
.star:hover { transform: scale(1.2); }  
</style>
```

Как это работает:

- Компонент реализует двустороннее связывание через **v-model**.
- **v-for** создает 5 звезд.
- Клик по звезде устанавливает рейтинг.

Постановка задачи: Менеджер книг для чтения

Нам предстоит реализовать приложение для управления списком книг со следующим функционалом:

- Добавление новых книг (название, автор, жанр).
- Отметка книг как прочитанных.
- Оценка прочитанных книг (от 1 до 5 звезд).
- Удаление книг.
- Фильтрация: все, непрочитанные, прочитанные.
- Поиск по названию или автору.
- Статистика: сколько книг прочитано, общее количество.
- Сохранение книг в localStorage.

Пошаговое руководство

Шаг 1: Создание проекта

1. Откройте терминал и выполните команду:

```
npm create vite@latest book-manager -- --template vue
```

2. Перейдите в папку проекта:

```
cd book-manager
```

3. Установите зависимости:

```
npm install
```

4. Очистите проект. В `src/App.vue` оставьте базовую структуру:

```
<template>
  <div>
    <h1>Менеджер книг</h1>
  </div>
</template>

<script setup>
</script>

<style>
* {
  margin: 0;
  padding: 0;
  box-sizing: border-box;
}
```

```
body {  
  font-family: Arial, sans-serif;  
  background: #f5f5f5;  
}  
</style>
```

5. Запустите сервер разработки:

```
npm run dev
```

Шаг 2: Создание компонента книги (BookCard.vue)

Создайте в папке `src/components` файл `BookCard.vue`:

```
<template>  
  <div class="book-card" :class="{ completed: book.completed }">  
    <div class="book-info">  
      <h3>{{ book.title }}</h3>  
      <p class="author">{{ book.author }}</p>  
      <span class="genre">{{ book.genre }}</span>  
    </div>  
  
    <div class="book-actions">  
      <div v-if="book.completed" class="rating">  
        <span  
          v-for="star in 5"  
          :key="star"  
          @click="$emit('rate', star)"  
        >  
          {{ star <= book.rating ? '★' : '☆' }}  
        </span>  
      </div>  
  
      <button  
        @click="$emit('toggle')"  
        :class="['btn', book.completed ? 'btn-secondary' : 'btn-primary']"  
      >  
        {{ book.completed ? 'Прочитано' : 'Отметить прочитанной' }}  
      </button>  
  
      <button @click="$emit('delete')" class="btn btn-danger">  
        ✕  
      </button>  
    </div>  
  </div>  
</template>  
  
<script setup>
```

```
defineProps(['book'])
defineEmits(['toggle', 'delete', 'rate'])
</script>

<style scoped>
.book-card {
  background: white;
  border-radius: 8px;
  padding: 16px;
  margin-bottom: 12px;
  box-shadow: 0 2px 4px rgba(0,0,0,0.1);
  display: flex;
  justify-content: space-between;
  align-items: center;
  transition: all 0.3s;
}

.book-card.completed {
  background: #f0f7f0;
  opacity: 0.8;
}

.book-info {
  flex: 1;
}

.book-info h3 {
  margin-bottom: 4px;
  color: #333;
}

.author {
  color: #666;
  font-size: 0.9em;
  margin-bottom: 4px;
}

.genre {
  background: #e0e0e0;
  padding: 2px 8px;
  border-radius: 4px;
  font-size: 0.8em;
}

.book-actions {
  display: flex;
  gap: 8px;
  align-items: center;
}

.rating {
  display: flex;
  gap: 2px;
}
```



```
.rating span {
  font-size: 20px;
  cursor: pointer;
  color: gold;
}

.rating span:hover {
  transform: scale(1.2);
}

.btn {
  padding: 8px 12px;
  border: none;
  border-radius: 4px;
  cursor: pointer;
  font-size: 0.9em;
  transition: background 0.3s;
}

.btn-primary {
  background: #4CAF50;
  color: white;
}

.btn-primary:hover {
  background: #45a049;
}

.btn-secondary {
  background: #2196F3;
  color: white;
}

.btn-secondary:hover {
  background: #1e87db;
}

.btn-danger {
  background: #f44336;
  color: white;
  padding: 8px 12px;
}

.btn-danger:hover {
  background: #da190b;
}
</style>
```

Что здесь происходит:

- Компонент отображает карточку книги с названием, автором и жанром.

- Для прочитанных книг показывается рейтинг звездочками.
- Кнопки позволяют отметить прочитанной, поставить оценку или удалить.
- Стили меняются в зависимости от статуса книги.

Шаг 3: Создание формы добавления (AddBookForm.vue)

Создайте файл `AddBookForm.vue`:

```
<template>
  <form @submit.prevent="handleSubmit" class="add-form">
    <h2>Добавить новую книгу</h2>

    <div class="form-group">
      <input
        v-model="formData.title"
        type="text"
        placeholder="Название книги"
        required
      />
    </div>

    <div class="form-group">
      <input
        v-model="formData.author"
        type="text"
        placeholder="Автор"
        required
      />
    </div>

    <div class="form-group">
      <select v-model="formData.genre" required>
        <option value="">Выберите жанр</option>
        <option value="Роман">Роман</option>
        <option value="Фантастика">Фантастика</option>
        <option value="Детектив">Детектив</option>
        <option value="Научная">Научная</option>
        <option value="Поэзия">Поэзия</option>
      </select>
    </div>

    <button type="submit" class="btn-submit">Добавить книгу</button>
  </form>
</template>

<script setup>
import { reactive } from 'vue'

const emit = defineEmits(['add-book'])

const formData = reactive({
  title: '',
```

```
    author: '',
    genre: ''
  })

  const handleSubmit = () => {
    emit('add-book', { ...formData })
    // Очистка формы
    formData.title = ''
    formData.author = ''
    formData.genre = ''
  }
</script>

<style scoped>
.add-form {
  background: white;
  padding: 20px;
  border-radius: 8px;
  box-shadow: 0 2px 4px rgba(0,0,0,0.1);
  margin-bottom: 20px;
}

.add-form h2 {
  margin-bottom: 15px;
  color: #333;
}

.form-group {
  margin-bottom: 15px;
}

.form-group input,
.form-group select {
  width: 100%;
  padding: 10px;
  border: 1px solid #ddd;
  border-radius: 4px;
  font-size: 1em;
}

.form-group input:focus,
.form-group select:focus {
  outline: none;
  border-color: #4CAF50;
}

.btn-submit {
  width: 100%;
  padding: 12px;
  background: #4CAF50;
  color: white;
  border: none;
  border-radius: 4px;
  font-size: 1em;
}
```

```
    cursor: pointer;
    transition: background 0.3s;
  }

  .btn-submit:hover {
    background: #45a049;
  }
</style>
```

Что здесь происходит:

- **reactive** создает реактивный объект для формы.
- **v-model** связывает поля формы с данными.
- **@submit.prevent** отменяет стандартную отправку формы.
- После добавления книги форма очищается.

Шаг 4: Создание фильтров и поиска (BookFilters.vue)

Создайте файл **BookFilters.vue**:

```
<template>
  <div class="filters">
    <div class="search">
      <input
        v-model="searchQuery"
        type="text"
        placeholder="Поиск по названию или автору..."
      />
    </div>

    <div class="filter-buttons">
      <button
        v-for="option in filterOptions"
        :key="option.value"
        @click="$emit('update:filter', option.value)"
        :class="['filter-btn', { active: filter === option.value }]"
      >
        {{ option.label }}
      </button>
    </div>

    <div class="stats">
      <p>Всего: {{ total }} | Прочитано: {{ completed }} | Осталось: {{ total -
        completed }}</p>
    </div>
  </div>
</template>

<script setup>
import { computed } from 'vue'
```

```
const props = defineProps(['filter', 'books'])
defineEmits(['update:filter'])

const searchQuery = defineModel('searchQuery')

const filterOptions = [
  { value: 'all', label: 'Все' },
  { value: 'unread', label: 'Непрочитанные' },
  { value: 'read', label: 'Прочитанные' }
]

const total = computed(() => props.books.length)
const completed = computed(() => props.books.filter(b => b.completed).length)
</script>

<style scoped>
.filters {
  background: white;
  padding: 20px;
  border-radius: 8px;
  box-shadow: 0 2px 4px rgba(0,0,0,0.1);
  margin-bottom: 20px;
}

.search {
  margin-bottom: 15px;
}

.search input {
  width: 100%;
  padding: 10px;
  border: 1px solid #ddd;
  border-radius: 4px;
  font-size: 1em;
}

.filter-buttons {
  display: flex;
  gap: 10px;
  margin-bottom: 15px;
}

.filter-btn {
  padding: 8px 16px;
  border: 1px solid #ddd;
  background: white;
  border-radius: 4px;
  cursor: pointer;
  transition: all 0.3s;
}

.filter-btn:hover {
  background: #f0f0f0;
}
```

```
.filter-btn.active {
  background: #4CAF50;
  color: white;
  border-color: #4CAF50;
}

.stats {
  padding-top: 15px;
  border-top: 1px solid #eee;
  color: #666;
  font-size: 0.9em;
}
</style>
```

Что здесь происходит:

- `defineModel` создает двустороннюю привязку для поискового запроса (новая фича Vue 3.4+).
- Фильтры подсвечиваются при выборе.
- Статистика обновляется автоматически через вычисляемые свойства.

Шаг 5: Сборка главного компонента (App.vue)

Теперь соберем всё вместе:

```
<template>
  <div class="app">
    <header>
      <h1>Менеджер книг</h1>
      <p>Управляй своей библиотекой</p>
    </header>

    <main>
      <AddBookForm @add-book="addBook" />

      <BookFilters
        v-model:searchQuery="searchQuery"
        v-model:filter="currentFilter"
        :books="books"
      />

      <div v-if="filteredBooks.length === 0" class="empty-state">
        <p>Книги не найдены :(</p>
        <p>Добавьте первую книгу или измените параметры поиска</p>
      </div>

      <div v-else class="books-list">
        <BookCard
          v-for="book in filteredBooks"
          :key="book.id"
          :book="book"
        />
      </div>
    </main>
  </div>
</template>
```

```
        @toggle="toggleBook(book.id)"
        @delete="deleteBook(book.id)"
        @rate="rateBook(book.id, $event)"
      />
    </div>
  </main>
</div>
</template>

<script setup>
import { ref, computed, watch } from 'vue'
import AddBookForm from './components/AddBookForm.vue'
import BookFilters from './components/BookFilters.vue'
import BookCard from './components/BookCard.vue'

// Состояние книг с загрузкой из localStorage
const books = ref([])

// Загрузка сохраненных книг
const savedBooks = localStorage.getItem('books')
if (savedBooks) {
  books.value = JSON.parse(savedBooks)
}

// Состояния фильтрации
const currentFilter = ref('all')
const searchQuery = ref('')

// Сохранение изменений
watch(books, (newBooks) => {
  localStorage.setItem('books', JSON.stringify(newBooks))
}, { deep: true })

// Добавление книги
const addBook = (bookData) => {
  const newBook = {
    id: Date.now(),
    ...bookData,
    completed: false,
    rating: 0
  }
  books.value.push(newBook)
}

// Переключение статуса
const toggleBook = (id) => {
  const book = books.value.find(b => b.id === id)
  if (book) {
    book.completed = !book.completed
    if (!book.completed) {
      book.rating = 0
    }
  }
}
}
```

```
// Оценка книги
const rateBook = (id, rating) => {
  const book = books.value.find(b => b.id === id)
  if (book && book.completed) {
    book.rating = rating
  }
}

// Удаление книги
const deleteBook = (id) => {
  if (confirm('Удалить книгу?')) {
    books.value = books.value.filter(b => b.id !== id)
  }
}

// Фильтрация и поиск книг
const filteredBooks = computed(() => {
  return books.value
    .filter(book => {
      if (currentFilter.value === 'unread') return !book.completed
      if (currentFilter.value === 'read') return book.completed
      return true
    })
    .filter(book => {
      if (!searchQuery.value) return true
      const query = searchQuery.value.toLowerCase()
      return book.title.toLowerCase().includes(query) ||
        book.author.toLowerCase().includes(query)
    })
})
</script>

<style>
* {
  margin: 0;
  padding: 0;
  box-sizing: border-box;
}

body {
  font-family: 'Segoe UI', Tahoma, Geneva, Verdana, sans-serif;
  background: #f0f2f5;
  line-height: 1.6;
}

.app {
  max-width: 800px;
  margin: 0 auto;
  padding: 20px;
}

header {
  text-align: center;
```



```
margin-bottom: 30px;
padding: 20px;
background: linear-gradient(135deg, #667eea 0%, #764ba2 100%);
color: white;
border-radius: 10px;
box-shadow: 0 4px 6px rgba(0,0,0,0.1);
}

header h1 {
  font-size: 2.5em;
  margin-bottom: 5px;
}

main {
  background: white;
  padding: 30px;
  border-radius: 10px;
  box-shadow: 0 2px 10px rgba(0,0,0,0.1);
}

.empty-state {
  text-align: center;
  padding: 40px;
  color: #999;
  font-size: 1.2em;
}

.empty-state p:first-child {
  font-size: 3em;
  margin-bottom: 20px;
}

.books-list {
  margin-top: 20px;
}
</style>
```

Ключевые моменты в App.vue:

- **ref** для реактивных данных.
- **watch** для сохранения в localStorage при каждом изменении.
- **computed** для фильтрации и поиска книг.
- Использование всех созданных компонентов.
- Двустороннее связывание с помощью **v-model** и **v-model:searchQuery**.

Шаг 6: Точка входа (main.js)

Убедитесь, что **main.js** выглядит так:

```
import { createApp } from 'vue'
import App from './App.vue'
```

```
createApp(App).mount('#app')
```

Дополнительные задания для самостоятельной работы

В качестве самостоятельной работы добавьте возможность отмечать книги как Избранные или Любимые.

Публикация на GitHub Pages

Шаг 1: Подготовка репозитория

1. Создайте новый репозиторий на GitHub.
2. Инициализируйте git в локальном проекте.

Шаг 2: Настройка Vite для GitHub Pages

Установите пакет `gh-pages`:

```
npm install --save-dev gh-pages
```

В файле `package.json` добавьте:

```
"homepage": "https://ваш-логин.github.io/book-manager/" // тут полная ссылка на  
Git Pages
```

В раздел `scripts` добавьте:

```
"predeploy": "npm run build",  
"deploy": "gh-pages -d dist"
```

Шаг 3: Настройка конфигурации Vite

Создайте или отредактируйте файл `vite.config.js`:

```
import { defineConfig } from 'vite'  
import vue from '@vitejs/plugin-vue'  
  
export default defineConfig({  
  plugins: [vue()],  
  base: '/book-manager/' // тут название репозитория  
})
```

Шаг 4: Деплой

Выполните команду:

```
npm run deploy
```

Шаг 5: Настройка GitHub Pages

1. Перейдите в настройки репозитория на GitHub (вкладка Settings).
2. В разделе Pages выберите ветку `gh-pages` и папку `/root`.
3. Сохраните — через несколько минут приложение будет доступно.

Что делать при обновлении приложения

При внесении изменений просто выполните:

```
npm run deploy
```

Изменения автоматически соберутся и загрузятся на GitHub Pages.

Формат отчёта

В область для загрузки (Практическое задание 14) прикрепите ссылку на GitHub Pages с работающим Vue-приложением. Страница должна быть проверена и работоспособна.